UNIVERZITET U NIŠU

ELEKTRONSKI FAKULTET U NIŠU

Master akademske studije

Računarstvo i informatika

Modul: Softversko inženjerstvo

Seminarski rad iz predmeta

Sistemi za upravljanje bazama podataka

Obrada transakcija, planovi izvršavanja transakcija, izolacija i zaključavanje u Firebird bazi podataka

Profesor:

dr Aleksandar S. Stanimirović

Student:

Jefimija Stamenović 1820/24

Mart, 2026. godine

Sadržaj

1. Uvod 4
   1.1. Koncept transakcije kao osnovnog stuba pouzdanosti informacionih sistema 4
   1.2. Izazovi paralelizma: Sukobi pri istovremenom pristupu podacima u višekorisničkom okruženju 4
   1.3. Važnost mehanizama izolacije za stabilnost poslovnih aplikacija 5
   1.4. Predmet i cilj rada: Analiza specifičnosti Firebird transakcionog modela i MGA arhitekture 5
2. ACID principi i nivoi izolacije 6
   2.1. Detaljna dekompozicija ACID svojstava 6
   2.2. Transakcione anomalije 7
      2.2.1. Prljavo čitanje 7
      2.2.2. Neponovljivo čitanje 7
      2.2.3. Fantomsko čitanje 7
   2.3. Nivoi izolacija transakcija 8
      2.3.1. Read Uncommitted 8
      2.3.2. Read Committed 8
      2.3.3. Repeatable Read 9
      2.3.4. Serializable 9
   2.4. Odnos nivoa izolacije i transakcionih anomalija 10
3. Multi-generational Architecture (MGA) 11
   3.1. Multi-Version Concurrency Control (MVCC) 11
   3.2. Mehanizam verzija zapisa: Kako se čuva istorija izmena bez blokiranja čitalaca? 11
   3.3. Copy-on-Write princip: Fizički smeštaj novih verzija podataka na stranice baze 12
   3.4. Transaction Inventory Page (TIP) 13
   3.5. Record Header 14
4. Implementacija nivoa izolacije u Firebird-u 15
   4.1. Snapshot 15
   4.2. Read Committed 16
      4.2.1. Razlika između RECORD VERSION i NO RECORD VERSION 16
   4.3. Snapshot Table Stability 17
   4.4. Read Committed Read Consistency 18

5. Mehanizmi zaključavanja (locking) i rešavanje konflikata 19
5.1. Pesimističko zaključavanje 19
5.2. Optimistički pristup 20
5.3. Arhitektura i uloga Lock Manager-a 20
5.4. Deadlock u Firebirdu: Algoritmi detekcije i strategije izbegavanja 21
5.5. Parametri čekanja: Wait, No Wait i uticaj Lock Timeout parametra 21
6. Održavanje baze i garbage collection 23
6.1. Problem rasta baze podataka (Database Bloat) usled akumulacije verzija zapisa 23
6.2. Analiza transakcionih markera 23
6.2.1. Definicija markera i njihova uloga 23
6.2.2. SQL monitoring markera u realnom vremenu 24
6.3. Strategije čišćenja: Cooperative, Background i Sweeping (Backsweep) procesi 26
7. Planovi izvršavanja (Execution Plans) i monitoring 28
7.1. Uloga TPB (Transaction Parameter Block) u donošenju odluka 28
7.2. Uticaj nivoa izolacije na izbor putanje 28
7.3. Dijagnostika transakcija u realnom vremenu putem sistemskih tabela (MON$TRANSACTIONS) 30
8. Zaključak 31
9. Literatura 32

1. Uvod

1.1. Koncept transakcije kao osnovnog stuba pouzdanosti informacionih sistema

U svetu savremenog razvoja softvera, gde se podaci generišu i menjaju u milisekundama, pitanje integriteta baze podataka prestalo je da bude samo tehnička karakteristika i postalo je osnova poslovne stabilnosti. Svaki put kada korisnik izvrši neku operaciju – bilo da je to prenos novca sa jednog računa na drugi, rezervacija avionske karte ili prosta izmena statusa narudžbenice u e-commerce sistemu – mi se oslanjamo na koncept transakcije.

Transakcija se najjednostavnije može definisati kao logička jedinica rada koja okuplja jednu ili više operacija nad podacima u neraskidivu celinu. Formalno, transakcija predstavlja logičku jedinicu rada koja obuhvata jedan ili više SQL upita koji se izvršavaju kao nedeljiva celina. To znači da, ili će se sve operacije unutar transakcije izvršiti uspešno ili neće nijedna.

Suština transakcionog rada nije samo u tome da se podaci upišu, već da se garantuje da će baza uvek ostati u stabilnom i predvidljivom stanju bez obzira na to da li je operacija uspela, da li je došlo do greške u softveru ili je server iznenada ostao bez napajanja. Bez snažnog mehanizma za upravljanje transakcijama, baza bi brzo postala skup kontradiktornih informacija, gde bi se, na primer, novac skinuo sa jednog računa, a nikada ne bi pojavio na drugom. Transakcija je taj "zaštitni omotač" koji osigurava da se ili sve planirane promene izvrše u potpunosti, ili da se, u slučaju bilo kakvog problema, baza vrati u stanje u kojem je bila pre početka procesa.

Transakcija prolazi kroz nekoliko faza – početak, izvršavanje čitanja i zapisa, a zatim se potvrđuje (COMMIT) ili poništava (ROLLBACK). Za robustan dizajn sistema je od preke važnosti eksplicitno rukovanje greškama i politika ponovnog pokušaja kod konflikata, kao i razumevanje nivoa izolacije koji određuju vidljivost promena između konkurenata

1.2. Izazovi paralelizma: Sukobi pri istovremenom pristupu podacima u višekorisničkom okruženju

Problem upravljanja transakcijama postaje eksponencijalno teži onog trenutka kada bazi pristupi više korisnika istovremeno. U idealnom, ali nerealnom svetu, transakcije bi se izvršavale jedna po jedna (serijski). Međutim, u realnim sistemima sa stotinama ili hiljadama paralelnih konekcija, performanse bi bile neprihvatljivo niske ako bismo naterali korisnike da čekaju jedan na drugog.

Zbog toga moderni sistemi, poput Firebird-a, moraju da podrže paralelizam (concurrency). Tu dolazimo do ključnog izazova: kako dozvoliti dvojici korisnika da menjaju isti podatak u isto vreme, a da pritom jedan drugom ne "pregaze" izmene ili, što je još češće, da jedan korisnik ne čita podatke koje je drugi tek počeo da menja, a možda će odustati od njihove izmene. Ovi sukobi nisu samo teoretska mogućnost, već su svakodnevica u razvoju aplikacija. Ako sistem za upravljanje bazom podataka (RDBMS) ne upravlja ovim sukobima na pametan način, onda dolazi do situacija gde se gube ažurirani podaci ili se donose pogrešne poslovne odluke na osnovu informacija koje su trenutno u statusu menjanja.

1.3. Važnost mehanizama izolacije za stabilnost poslovnih aplikacija

Izolacija je deo transakcione logike koji određuje koliko jedna transakcija sme da "vidi" ono što druge transakcije trenutno rade. To je zapravo balansiranje između dve suprotstavljene potrebe: brzine rada i tačnosti podataka. Što je izolacija stroža, to su podaci sigurniji, ali je sistem sporiji jer transakcije moraju više da čekaju ili da troše više resursa na proveru verzija.

Za programere i arhitekte baza podataka, razumevanje izolacije je kritično jer direktno utiče na ponašanje aplikacije u kriznim momentima. Loše odabran nivo izolacije može dovesti do toga da aplikacija radi savršeno tokom testiranja sa jednim korisnikom, ali da počne da izbacuje nasumične greške ili prikazuje pogrešne rezultate čim se broj korisnika poveća. Stabilna poslovna aplikacija ne sme da zavisi od sreće u redosledu pristupa podacima, već mora biti projektovana tako da transakcioni mehanizam baze automatski rešava ovakve konflikte, ostavljajući programeru fokus na poslovnu logiku, a ne na ručno zaključavanje tabela.

1.4. Predmet i cilj rada: Analiza specifičnosti Firebird transakcionog modela i MGA arhitekture

Iako svi veći sistemi za baze podataka teže istim ciljevima, načini na koji ih postižu se drastično razlikuju. Firebird se u ovom domenu izdvaja svojom specifičnom arhitekturom pod nazivom **Multi-Generational Architecture (MGA)**. Za razliku od nekih drugih popularnih sistema koji koriste komplikovane logove da bi "vratili film" unazad u slučaju greške, Firebird koristi princip verzija zapisa.

Cilj ovog rada je da se detaljno istraži kako Firebird upravlja transakcijama, kako njegovi specifični planovi izvršavanja utiču na brzinu obrade i na koji način mehanizmi zaključavanja sprečavaju konflikte. Poseban fokus biće na razumevanju toga kako Firebird postiže da čitaoci podataka nikada ne blokiraju pisce i obrnuto, a to je ujedno i jedna od njegovih najznačajnijih prednosti. Kroz analizu unutrašnjih struktura poput TIP-a (Transaction Inventory Page) i različitih nivoa izolacije, ovaj rad će pokušati da pruži jasan uvid u to kako se teoretski principi transakcija pretvaraju u praktično, robusno i efikasno rešenje unutar Firebird ekosistema.

2. ACID principi i nivoi izolacije

Da bismo razumeli specifičnost Firebird baze podataka, neophodno je da prvo definišemo standard na kojima leži svaki sistem za upravljanje bazama podataka. Upravljanje transakcijama poštuje strogo definisan skup pravila koji garantuju da će podaci preživeti čak i najteže sistemske greške.

2.1. Detaljna dekompozicija ACID svojstava

Skraćenica **ACID** predstavlja četiri ključna svojstva na kojima počiva pouzdanost svake moderne baze podataka. Svako od sledećih svojstava rešava specifičan problem koji se javlja u radu sa podacima:

1) **Atomičnost (engl. Atomicity) –** posmatra transakciju kao operaciju „sve ili ništa". Baza ne sme dozvoliti „polovične" rezultate koji bi narušili logičku celinu podataka. Konkretan primer bi bio prenos novca sa računa A na račun B. Transakcija se u ovom slučaju sastoji od dve operacije: skidanja iznosa sa računa A i dodavanja istog iznosa na račun B. Ako sistem otkaže nakon prve operacije, novac bi nestao iz sistema. Atomičnost garantuje da će se u tom slučaju prva operacija poništiti (*rollback*) i da će se baza vratiti u početno stanje.
2) **Konzistentnost (engl. Consistency) –** osigurava da transakcija prevodi bazu iz jednog validnog stanja u drugo validno stanje pri čemu ne sme doći do kršenja nekog pravila integriteta kao što je strani ključ ili definisano ograničenje. Na primer, ako u bazi postoji pravilo da stanje na računu ne sme biti negativno, tj. manje od 0, transakcija koja pokušava da isplati više novca nego što ima na računu biće automatski prekinuta. Baza odbija da prihvati promenu koja bi narušila ovo pravilo čime čuva logičku ispravnost podataka.
3) **Izolacija (engl. Isolation)** – ovo je možda i najkompleksnije svojstvo u višekorisničkom radu. Ono garantuje da operacije koje se izvršavaju istovremeno neće uticati jedna na drugu na nepredviđen način. Svaka transakcija treba da se „oseća" kao da je jedina u sistemu. Stepen te "privatnosti" varira u zavisnosti od nivoa izolacije koji odaberemo (što je ključna tema ovog rada). Ako imamo jednu transakciju koja računa ukupnu sumu svih faktura u bazi (što traje nekoliko sekundi) i drugu transakciju koja u istom trenutku menja iznos na nekoj fakturi,onda će izolacija obezbediti da prva transakcija dobije tačan rezultat (ili stanje pre ili stanje posle izmene), a ne neku nasumičnu, polovičnu sumu.
4) **Trajnost (engl. Durability):** Kada sistem potvrdi da je transakcija uspešno završena (Commit), te promene postaju trajne i nepromenljive. Čak i ako server padne sekundu nakon potvrde, podaci moraju biti bezbedno sačuvani na disku ili u logovima tako da se mogu rekonstruisati pri ponovnom pokretanju. Na primer, nakon što korisnik klikne na „Sačuvaj" i dobije potvrdu od baze, on može biti siguran da su podaci upisani na disk. Čak i ako bi se server fizički ugasio (nestanak struje) delić sekunde kasnije, podaci će biti tu kada se sistem ponovo pokrene, jer je baza već osigurala njihov trajni zapis.

2.2. Transakcione anomalije

Kada se u sistemu omogući paralelni rad više korisnika nad istim skupom podataka, a pritom se ne koristi maksimalni nivo izolacije, neminovno dolazi do sukoba između performansi i tačnosti. Sekvencijalno izvršavanje transakcija bi bilo najsigurnije, ali bi u praksi dovelo do zagušenja baze. Zato se u realnim sistemima dopušta preplitanje transakcionih operacija, što sa sobom povlači rizik od pojave **transakcionih anomalija**. Suština ovih anomalija je u tome što jedna transakcija može postati svesna promena koje druga transakcija vrši pre nego što su te promene konačno potvrđene. U poslovnim aplikacijama, ovakva narušavanja izolacije mogu dovesti do ozbiljnih grešaka u podacima, kao što su netačni finansijski bilansi ili prodaja robe koja je u tom trenutku već procesuirana u drugom nalogu.

2.2.1. Prljavo čitanje

Do **prljavog čitanja (engl. Dirty Read)** dolazi kada jedna transakcija čita podatke koje je druga transakcija izmenila, ali još uvek nije potvrdila izmene tj. nije urađen *commit*. Problem nastaje ako druga transakcija na kraju odustane od izmena tj. uradi *rollback*. U tom slučaju, prva transakcija je pročitala "prljave" podatke, radi sa podacima koji su bili privremeni i koji formalno nikada nisu postali deo baze. Ovo je najopasnija anomalija i većina modernih baza, uključujući Firebird, je podrazumevano ne dozvoljava.

Primer: Prodavac u salonu automobila menja cenu vozila sa 20.000 na 15.000 evra, ali još nije potvrdio unos. U tom trenutku, kupac putem aplikacije vidi sniženu cenu i pokreće proces kupovine. Ukoliko prodavac uoči grešku i poništi transakciju, kupac je praktično doneo odluku na osnovu cene koja je bila „prljava” i nepostojeća.

2.2.2. Neponovljivo čitanje

Neponovljivo čitanje (engl. Non-repeatable Read) je anomalija koja nastaje kada jedna transakcija u toku svog trajanja dva puta čita isti red podataka, ali dobija različite rezultate. Razlog je taj što je između dva čitanja neka druga transakcija izmenila taj red i uspešno potvrdila promenu. Iako su podaci koje transakcija čita validni, ona gubi svoju unutrašnju konzistentnost jer se stanje objekta promenilo usred procesa obrade.

Primer: Program za generisanje izveštaja očita dug klijenta od 1.000 dinara. Dok proces pripreme izveštaja traje, klijent na drugom mestu izvrši uplatu i ta promena se proknjiži. Ako program pre same finalizacije ponovo proveri isto polje, videće nulu. Rezultat je dokument koji sadrži kontradiktorne vrednosti za istu stavku.

2.2.3. Fantomsko čitanje

Fantomsko čitanje (engl. Phantom Read) je specifičan oblik gde se ne menja vrednost postojećeg reda već se menja čitav skup podataka koji zadovoljava određeni uslov. To se dešava kada se u toku trajanja jedne transakcije pojave novi zapisi koji odgovaraju istom kriterijumu pretrage.

Primer: Menadžer zatraži listu svih zaposlenih sa platom većom od 100.000 dinara i dobije spisak od 10 imena. Dok on pregleda tu listu, kadrovska služba unese novog radnika sa platom od 110.000

dinara i potvrdi unos. Kada menadžer u okviru iste transakcije pokrene komandu za izračunavanje ukupnog budžeta, dobiće sumu za 11 ljudi, iako mu je početna lista prikazala deset.

2.3. Nivoi izolacija transakcija

Nivoi izolacije predstavljaju mehanizam kojim baza podataka kontroliše vidljivost promena između transakcija koje se izvršavaju u isto vreme. Njihova glavna uloga je sprečavanje anomalija, ali uz svesno prihvatanje kompromisa—što je izolacija stroža, to je konzistentnost podataka veća, ali su performanse lošije jer sistem troši više resursa na zaključavanja i proveru verzija. Standardni model prepoznaje četiri nivoa: *Read Uncommitted*, *Read Committed*, *Repeatable Read* i *Serializable*. Važno je istaći da Firebird, zahvaljujući svojoj specifičnoj arhitekturi verzija zapisa (**MGA – Multi-Generational Architecture**), ove nivoe ne rešava na klasičan način putem fizičkog zaključavanja redova. Umesto da čitaoci blokiraju pisce, Firebird svakoj transakciji isporučuje odgovarajuću verziju podatka. To omogućava bazi da postigne visoku izolaciju bez drastičnog pada brzine, o čemu će biti više reči u poglavlju o unutrašnjoj arhitekturi.

2.3.1. Read Uncommitted

Read Uncommitted predstavlja najslabiji nivo izolacije gde je fokus gotovo isključivo na performansama, dok se integritet podataka stavlja u drugi plan. Glavna karakteristika ovog nivoa je da transakcija može da pristupi izmenama koje su drugi korisnici izvršili, ali ih još uvek nisu zvanično potvrdili putem **COMMIT** naredbe. Zbog ovakve popustljivosti, ovde su teorijski dozvoljene sve poznate transakcione anomalije, a naročito prljavo čitanje što ovaj nivo čini neprikladnim za finansijske i druge kritične poslovne sisteme. Upravo zbog visokog rizika po konzistentnost, moderni DBMS sistemi, uključujući i Firebird, ovaj nivo često uopšte ne podržavaju u izvornom obliku ili ga automatski prebacuju na stroži nivo izolacije. Njegova primena je danas svedena na specifične slučajeve gde su podaci informativnog karaktera i gde povremena netačnost ne izaziva sistemski problem, kao što je prikupljanje telemetrijskih podataka ili očitavanje vrednosti sa senzora u realnom vremenu.

2.3.2. Read Committed

Read Committed je najrasprostranjeniji nivo izolacije u produkcionim okruženjima jer nudi optimalan balans između brzine rada i sigurnosti podataka. Osnovno pravilo ovog nivoa je da transakcija može videti isključivo one promene koje su druge transakcije uspešno potvrdile pre samog trenutka čitanja. Time se efikasno eliminiše pojava prljavog čitanja jer korisnik nikada ne dolazi u kontakt sa privremenim ili neproverenim stanjima baze. Ipak, ovaj nivo ne garantuje stabilnost podataka tokom celog životnog ciklusa transakcije. On štiti od čitanja tuđih radnih verzija, ali ne sprečava situaciju da se podaci promene u intervalu između dva upita unutar iste sesije. Zbog toga je pogodan za sisteme gde je važno da se vidi trenutno, realno stanje baze, uz svest o tome da se konzistentnost na nivou ponovljenih upita žrtvuje radi veće propusne moći sistema.

2.3.3. Repeatable Read

Repeatable Read je nivo izolacije koji garantuje da će ponovna čitanja istih redova unutar jedne transakcije vratiti identične vrednosti, bez obzira na to što druge transakcije u međuvremenu vrše i potvrđuju izmene nad tim istim podacima. Ovaj nivo eliminiše anomalije prljavog i neponovljivog čitanja pružajući korisniku stabilan pogled na podatke tokom čitavog procesa obrade. U sistemima kao što je Firebird, ovaj nivo se implementira kroz **SNAPSHOT** režim rada, gde svaka transakcija operiše sa logičkom slikom baze koja je nastala u trenutku njenog pokretanja. Sve naknadne modifikacije drugih korisnika ostaju nevidljive za trenutnu transakciju, čime se postiže visoka konzistentnost, ali uz cenu povećane potrošnje memorije zbog čuvanja starih verzija zapisa. Prednost ovog nivoa je u tome što omogućava dugotrajne operacije, poput generisanja kompleksnih izveštaja, bez rizika da će se podaci promeniti "u hodu" i dovesti do kontradiktornih rezultata. Ipak, standardna definicija ovog nivoa izolacije i dalje ostavlja mogućnost pojave fantomskog čitanja gde se broj redova koji ispunjavaju određeni kriterijum može promeniti ako druga transakcija ubaci nove zapise. Zbog specifičnosti MGA arhitekture, Firebird ovaj problem rešava efikasnije od većine drugih DBMS-ova, ali se generalno smatra da je Repeatable Read korak ispod potpune serijalizacije jer se fokusira na očuvanje stanja već pročitanih redova.

2.3.4. Serializable

Serializable predstavlja najstroži nivo izolacije koji obezbeđuje potpunu izolaciju tako što simulira sekvencijalno izvršavanje transakcija. U ovom režimu, baza garantuje da će krajnji rezultat paralelnog rada više korisnika biti identičan onome koji bi se dobio da su se te transakcije izvršavale jedna po jedna, a ne istovremeno. Ovaj nivo eliminiše apsolutno sve tipove anomalija, uključujući prljavo, neponovljivo i fantomsko čitanje. Za razliku od nižih nivoa, Serializable ne samo da "zamrzava" pogled na postojeće podatke, već aktivno štiti integritet čitavog skupa podataka koji je transakcija obradila, sprečavajući bilo kakve međusobne uticaje koji bi narušili logičku doslednost sistema.

U praksi se ovaj nivo realizuje na dva osnovna načina:

- **Pesimistički pristup –** podrazumeva korišćenje strogih zaključavanja nad čitavim opsezima tabela ili indeksnim ključevima, čime se fizički sprečavaju drugi korisnici da vrše izmene.
- **Optimistički pristup** – pristup bliži Firebird filozofiji i nasuprot pesimističkom pristupu, on dozvoljava rad, ali vrši strogu proveru konflikata u trenutku potvrđivanja izmena.

Cena ovakve sigurnosti je značajno smanjen paralelizam i povećana latencija jer sistem troši dodatne resurse na validaciju svakog koraka. Zbog toga se ovaj nivo koristi isključivo za najkritičnije operacije, kao što su složeni finansijski transferi gde je očuvanje apsolutnog integriteta podataka prioritet u odnosu na brzinu odziva.

2.4. Odnos nivoa izolacije i transakcionih anomalija

Sada kada smo definisali standardne nivoe izolacije, možemo pogledati tablicu u kojoj vidimo koje određene anomalije ne mogu da se pojave na određenom nivou izolacije:

Anomalija	READ UNCOMMITTED	READ COMMITTED	REPEATABLE READ	SERIALIZABLE
Dirty read	Da	Ne	Ne	Ne
Non-repeatable read	Da	Da	Ne	Ne
Phantom read	Da	Da	Ne	Ne
Lost update	Da	Da	Da	Ne
Write skew	Da	Da	Da	Ne

Tabela 1:Anomalije vs Standardni nivoi izolacije

3. Multi-generational Architecture (MGA)

Za razliku od tradicionalnih relacionih baza koje integritet podataka održavaju komplikovanim sistemima zaključavanja (engl. *locking*), Firebird se oslanja na specifičan koncept poznat kao **Multi-Generational Architecture (MGA)**. Osnovna filozofija ovog pristupa je da baza podataka u svakom trenutku čuva više verzija (generacija) istog zapisa. Ovakva struktura direktno rešava problem konkurentnosti, omogućavajući sistemu da opsluži ogroman broj istovremenih korisnika bez međusobnog blokiranja procesa čitanja i upisa.

3.1. Multi-Version Concurrency Control (MVCC)

Teorijski temelj na kojem počiva MGA je **Multi-Version Concurrency Control (MVCC)**. Glavna ideja ovog modela je eliminacija čekanja i to na sledeći način – umesto da transakcija koja želi da izmeni podatak "zaključa" taj red i primora sve ostale korisnike na čekanje, sistem kreira novu verziju tog istog podatka. U praktičnom smislu, ovo znači da čitaoci (transakcije koje samo pregledaju podatke) nikada ne čekaju pisce (transakcije koje menjaju podatke). Kada čitalac zatraži podatak koji je trenutno u procesu izmene od strane drugog korisnika, Firebird mu isporučuje poslednju potvrđenu (*committed*) verziju koja je postojala u trenutku kada je čitalac započeo svoju transakciju. Time se uklanja jedno od najvećih uskih grla u arhitekturi baza podataka – blokiranje resursa usled konflikta između procesa analize i procesa ažuriranja podataka.

3.2. Mehanizam verzija zapisa: Kako se čuva istorija izmena bez blokiranja čitalaca?

U Firebirdu, operacija `UPDATE` ne funkcioniše po principu prepisivanja podataka na istoj fizičkoj lokaciji (tzv. *in-place update*). Svaki put kada se izvrši izmena nad određenim podacima, Firebird kreira novu fizičku verziju zapisa, dok prethodna verzija ostaje netaknuta. Rezultat ovakvog pristupa je da svaki logički red u tabeli zapravo predstavlja vremensku seriju verzija koje su međusobno povezane u lanac. Ovaj lanac funkcioniše po principu jednostruko povezane liste, gde najnovija verzija zapisa u svom zaglavlju nosi pokazivač (engl. *Back Interest Pointer*) na svoju neposrednu prethodnu verziju.

Kada *engine* pristupi određenom redu, on započinje proces validacije verzije kako bi odredio koji podatak je relevantan za datog korisnika. Firebird prolazi kroz lanac verzija i traži onu verziju koja je bila u stanju "*committed*" u trenutku startovanja tekuće transakcije. Sve verzije koje su kreirale transakcije sa statusom "*active*" (one koje su još uvek u toku) ili "*rolled back*" (one koje su poništene), sistem jednostavno ignoriše. Detaljan algoritam za navigaciju kroz ovaj lanac bi izgledao ovako:

1) **Pozicioniranje na vrh** – baza uvek prvo pristupa najnovijoj verziji zapisa koja se nalazi na stranici podataka.
2) **Provera statusa** – *engine* očitava *Transaction ID* (TID) iz zaglavlja te verzije i proverava status te transakcije na TIP stranici.
3) **Evaluacija vidljivosti** – na osnovu nivoa izolacije (npr. *Snapshot*), baza odlučuje da li je ta verzija "vidljiva". Ako je transakcija koja je kreirala tu verziju još uvek aktivna ili je poništena, ta verzija se odbacuje.

4) **Navigacija unazad** – ukoliko najnovija verzija nije validna za trenutnog korisnika, baza prati *Back Interest Pointer* i prelazi na prethodnu verziju u lancu. Ovaj proces se ponavlja sve dok se ne pronađe generacija zapisa koja je bila potvrđena u trenutku kada je korisnikova transakcija započela svoj rad.

Ovakav mehanizam omogućava da različite transakcije istovremeno vide različite verzije istog reda, što je ključno za postizanje izolacije bez upotrebe katanaca (engl. *locks*) na samim podacima. Čitaoci nikada ne blokiraju pisce, jer se uvek mogu "vratiti kroz vreme" do verzije koja im je potrebna.

Primer: Zamislimo tabelu sa zalihama gde artikal ima količinu 100. Transakcija T1 (pisac) menja količinu na 80, ali još nije uradila *commit*. U tom trenutku, transakcija T2 (čitalac) pokreće upit. Firebird u lancu verzija vidi novu verziju sa vrednošću 80, ali proverom statusa transakcije T1 na TIP stranici utvrđuje da ona još nije potvrđena. Prateći *Back Interest Pointer*, čitalac T2 dolazi do prethodne verzije sa vrednošću 100 i nju isporučuje korisniku. Čim T1 uradi *commit*, svaka nova transakcija će odmah videti vrednost 80, dok će stare transakcije koje su još uvek aktivne i dalje čitati 100, čuvajući na taj način konzistentnost svog pogleda na bazu..

3.3. Copy-on-Write princip: Fizički smeštaj novih verzija podataka na stranice baze

Fizička implementacija MGA sistema u Firebirdu se zasniva na **Copy-on-Write** (kopiraj pri upisu) principu. U tradicionalnim bazama podataka, operacija **UPDATE** često podrazumeva prepisivanje starog podatka novim na istoj fizičkoj adresi na disku. U Firebirdu, međutim, originalni podatak se nikada ne menja "u mestu". Svaka modifikacija zapravo pokreće proces kreiranja nove fizičke kopije tog zapisa sa ažuriranim vrednostima, dok stara verzija ostaje netaknuta na svojoj prvobitnoj lokaciji.

Kada dođe do izmene, *engine* prvo locira slobodan prostor na stranicama podataka. Firebird teži optimizaciji tako što pokušava da novu verziju zapisa smesti na istu stranicu gde se nalazi i prethodna verzija sa ciljem da se smanji broj potrebnih ulazno-izlaznih (I/O) operacija prilikom čitanja lanca verzija. Ako na toj stranici nema dovoljno mesta, onda se nova verzija smešta na prvu sledeću slobodnu stranicu.

Ono što ovaj proces čini specifičnim jeste način na koji se održava veza između verzija. Stari zapis ostaje na svom fizičkom mestu, ali se njegovo zaglavlje ažurira tako da on postane "istorijski" zapis koji pokazuje na novu, aktuelniju verziju. Ovaj pristup ima nekoliko ključnih prednosti:

- **Brzina upisa** – baza ne gubi vreme na kompleksno pomeranje postojećih podataka ili reorganizaciju indeksa u realnom vremenu; ona jednostavno dodaje novu verziju na kraj lanca verzija.
- **Trenutni Rollback** – ukoliko se transakcija koja je vršila izmenu otkaže (*rollback*), Firebird ne mora da radi nikakvu fizičku restauraciju podataka. Stari podaci su i dalje tu, netaknuti, a nova, neuspešna verzija će jednostavno biti ignorisana i kasnije uklonjena.
- **Konkurentnost** – pošto se stari podaci ne prepisuju, transakcije koje su počele pre izmene mogu neometano da čitaju originalne vrednosti bez potrebe za privremenim tabelama ili *undo* logovima.

Da bi bilo jasnije, zamislimo situaciju gde se zapis nalazi na *stranici 100*, na *slotu 5*. Kada korisnik promeni taj zapis, Firebird će na slobodnom mestu (npr. *slot 12* na istoj stranici) upisati novu verziju. Zaglavlje na *slotu 5* će sada sadržati informaciju da je taj zapis postao prethodna verzija i "uputiće" svaki novi upit na *slot 12*. Na taj način, fizička struktura baze raste sa svakom izmenom, što se kasnije rešava procesom sakupljanja otpada (*Garbage Collection*), kako bi se sprečilo nepotrebno trošenje prostora na disku (*Database Bloat*).

3.4. Transaction Inventory Page (TIP)

Da bi Firebird u svakom trenutku mogao da donese odluku o tome koju verziju zapisa sme da prikaže kojoj transakciji, on mora da se osloni na specijalizovanu strukturu pod nazivom **Transaction Inventory Page (TIP)**. Ova stranica funkcioniše kao centralni, globalni registar statusa svih transakcija koje su ikada pokrenute u bazi (ili barem onih koje još uvek imaju relevantne verzije zapisa).

Fizički, TIP je sistemska stranica koja se sastoji od niza bitova. Svakoj transakciji, čim se pokrene, dodeljuje se jedinstveni, monotono rastući redni broj – **Transaction ID (TID)**. Na TIP stranici, za svaki TID su rezervisana tačno **dva bita** koja u binarnom formatu definišu trenutni status te transakcije. Ova dva bita mogu imati četiri kombinacije od kojih svaka nosi bitnu informaciju za mehanizam izolacije:

- **00 (Active)** – ovo je početno stanje svake transakcije. To znači da je transakcija pokrenuta, da možda trenutno vrši izmene, ali još uvek nije ni potvrđena (commit) ni poništena (rollback). Sve verzije zapisa koje nose ovaj TID su nevidljive za sve ostale korisnike u sistemu (osim za samu tu transakciju).
- **01 (Committed)** – ovaj status označava da je transakcija uspešno završena i da su njene izmene postale trajni deo baze. Tek kada se status na TIP stranici promeni u 01, te verzije zapisa postaju kandidati za čitanje od strane drugih, novijih transakcija.
- **10 (Rolled back)** – ukoliko korisnik poništi transakciju ili dođe do pada aplikacije, status se postavlja na 10. Umesto toga, on ih jednostavno "markira" kao nepostojeće putem ovog statusa. Prilikom sledećeg čitanja, *engine* će videti ovaj status i automatski preskočiti te verzije.
- **11 (Limbo)** — Ovo stanje se javlja isključivo kod distribuiranih transakcija (*Two-Phase Commit*). Ono signalizira da je lokalni deo posla završen, ali se čeka potvrda od spoljnog koordinatora pre nego što se transakcija konačno proglasi potvrđenom ili poništenom.

Kada upit (npr. SELECT) dođe do nekog reda na stranici podataka, on prvo "pročita" TID iz zaglavlja tog reda. Zatim, brzim pristupom u memoriju (gde se TIP stranice obično keširaju), proverava status tog TID-a. Ako je status na TIP stranici 01 (*commtted*), podatak se prihvata. Ako je 00 ili 10, engine prati lanac verzija unazad (preko *Back Interest* pointera) sve dok ne nađe TID koji ima status 01.

3.5. Record Header

Svaki zapis (red) u Firebird bazi podataka, pored korisničkih kolona koje definišu šemu tabele, sadrži i fiksni, nevidljivi deo koji se naziva **Record Header** (zaglavlje zapisa). Ovo zaglavlje predstavlja kontrolni centar za MGA mehanizam jer upravo u njemu *engine* pronalazi ključne metapodatke neophodne za navigaciju kroz verzije i određivanje vidljivosti podataka u višekorisničkom okruženju.

Struktura zaglavlja zapisa u Firebirdu sadrži nekoliko elemenata koji omogućavaju funkcionisanje MVCC modela:

- **Transaction ID (TID) – p**redstavlja redni broj transakcije koja je kreirala tu specifičnu verziju reda. Ovaj identifikator je direktno povezan sa TIP stranicom; kada engine pročita TID iz zaglavlja, on odmah "zna" koju poziciju na TIP stranici treba da proveri kako bi utvrdio da li je ta verzija potvrđena, aktivna ili poništena.
- **Back Interest Pointer –** ovo je fizički pokazivač koji vodi do prethodne verzije tog istog reda na stranicama podataka. Ukoliko trenutna verzija zapisa nije validna za transakciju koja vrši čitanje, *engine* koristi ovaj pokazivač kako bi se vratio jedan korak unazad u istoriji izmena. Na taj način se formira lanac verzija koji omogućava rekonstrukciju stanja baze za bilo koju aktivnu transakciju.
- **Record Flags – s**kup bitova koji definišu stanje samog zapisa na stranici kao što su informacije o tome da li je zapis fragmentisan (ako ne može da stane na jednu stranicu) ili da li predstavlja najnoviju verziju u lancu.

Zahvaljujući ovim metapodacima, Firebird može veoma brzo da donese odluku o isporuci podataka korisniku bez potrebe za eksternim tabelama zaključavanja. Kada upit stigne do određenog fizičkog slota na stranici, on prvo analizira **TID** iz zaglavlja. Upoređivanjem tog broja sa stanjem na **TIP stranici** i uzimajući u obzir nivo izolacije u kojem transakcija radi (npr. *Snapshot* ili *Read Committed*), engine momentalno odlučuje: ili se podatak čita direktno sa te lokacije, ili se prati **Back Interest Pointer** ka starijoj generaciji.

Primer: Ukoliko transakcija sa brojem 600 pokuša da pročita red čije zaglavlje nosi TID 605, engine će uvideti da je taj podatak kreirala transakcija koja je počela nakon nje. Bez obzira na to da li je transakcija 605 već uradila *Commit*, transakcija 600 je "ne sme" videti u *Snapshot* režimu. Engine tada koristi pokazivač iz zaglavlja verzije 605, "skače" na lokaciju prethodne verzije (npr. one sa TID 580) i proverava njenu vidljivost. Ovaj proces se odvija transparentno i munjevito, osiguravajući da svaki korisnik uvek ima konzistentan pogled na podatke, bez obzira na intenzitet izmena koje vrše drugi učesnici u sistemu.

4. Implementacija nivoa izolacije u Firebird-u

Izolacija transakcija u Firebirdu nije rigidna, fiksna kategorija, već fleksibilan sistem koji se prilagođava specifičnim potrebama poslovnih procesa. Dok određeni procesi zahtevaju apsolutnu digitalnu preciznost i nepoverljivost prema paralelnim izmenama (poput izrade završnog računa), drugi primarno zahtevaju uvid u najsvežije dostupne podatke u realnom vremenu (poput kontrole zaliha u maloprodaji). Firebird ove varijabilne zahteve rešava kroz tri osnovna nivoa izolacije koji direktno upravljaju načinom na koji motor baze interpretira lance verzija zapisa.

4.1. Snapshot

Snapshot je nivo izolacije koji korisniku pruža "zamrznutu sliku" baze podataka, osiguravajući potpuno dosledan pogled na podatke u trenutku startovanja transakcije. Onog momenta kada transakcija započne rad, Firebird beleži stanje TIP stranice, odnosno identifikuje broj poslednje potvrđene transakcije. Od tog trenutka, pa sve do samog završetka, transakcija živi u svojevrsnom izolovanom vremenskom mehuru — ona vidi bazu tačno onakvu kakva je bila u sekundi njenog pokretanja.

Čak i ako desetine drugih korisnika u međuvremenu izmene i potvrde milione zapisa, Snapshot transakcija će te promene ignorisati. Ona će za svaki traženi red uporno pratiti lanac verzija unazad, sve dok ne pronađe generaciju koja je bila potvrđena pre njenog "rođenja". Ovakav pristup je nezamenljiv kod dugačkih i kompleksnih izveštaja koji zahtevaju unutrašnju logičku konzistentnost.

Primer: Korišćenjem nivoa izolacije SNAPSHOT READ ONLY, sesija A fiksira stanje baze podataka u trenutku pokretanja transakcije, kreirajući svojevrsnu "sliku" podataka.

Izvorni kod: Snapshot Read Only

```
1  /* sesija A */
2  SET TRANSACTION ISOLATION LEVEL SNAPSHOT READ ONLY;
3  SELECT E.EMP_NO, E.FIRST_NAME, E.LAST_NAME, D.DEPARTMENT, E.SALARY
4  FROM EMPLOYEE E
5  JOIN DEPARTMENT D ON E.DEPT_NO = D.DEPT_NO
6  WHERE E.SALARY > 80000
7  ORDER BY E.SALARY DESC;
8
9  /*sesija B*/
10 UPDATE EMPLOYEE SET SALARY = 7380000 WHERE EMP_NO = 118;
11 COMMIT;
12 /*ponovo izvršiti SELECT iz sesije A*/
```

Rezultat: Prilikom prvog izvršavanja SELECT upita u sesiji A, zaposleni gde je EMP_NO = 118 je imao 7480000. U međuvremenu je izvršena sesija B koja je promenila ovu vrednost, ali to nije uticalo na promenu vrednosti u sesiji A prilikom ponovnog izvršavanja SELECT-a. Dakle, čak i kada sesija B u međuvremenu izvrši promenu plate zaposlenog i potvrdi je komandom COMMIT, sesija A će prilikom ponovnog čitanja videti isključivo stare vrednosti.

4.2. Read Committed

Za razliku od statične prirode Snapshot-a, **Read Committed** nivo je znatno dinamičniji i prilagođeniji transakcionim sistemima visoke frekvencije. Ovaj nivo dozvoljava transakciji da "vidi" promene koje su druge transakcije potvrdile čak i nakon što je ona sama počela sa radom. Na taj način se postiže optimalan balans između ažurnosti informacija i performansi sistema. Kad god se izvrši novi SQL upit unutar ovakve transakcije, Firebird ponovo konsultuje TIP stranicu i isporučuje najnovije potvrđene verzije zapisa koje su dostupne u tom trenutku.

4.2.1. Razlika između RECORD VERSION i NO RECORD VERSION

Unutar Read Committed nivoa, Firebird nudi dve ključne pod-varijante koje suštinski menjaju ponašanje baze u situacijama kada dođe do pokušaja pristupa podacima koji se trenutno menjaju:

- **RECORD VERSION** – ukoliko transakcija A pokuša da pročita red nad kojim transakcija B trenutno vrši izmenu, Firebird neće zaustaviti proces. Umesto čekanja, on će trenutno "posegnuti" u lanac verzija i isporučiti poslednju potvrđenu generaciju tog reda. Ovo omogućava maksimalnu propusnu moć sistema jer čitaoci nikada ne čekaju na pisce.
- **NO RECORD VERSION** – u ovom režimu, ako je red koji korisnik želi da pročita u procesu izmene (odnosno postoji njegova novija, još uvek nepotvrđena verzija), transakcija će pauzirati svoj rad. Ona ulazi u stanje čekanja sve dok se ta druga transakcija ne završi (commit ili rollback). Tek po razrešenju konflikta, korisnik dobija najnoviji, upravo potvrđeni podatak. Ovaj režim se koristi u kritičnim scenarijima gde je imperativ raditi sa apsolutno poslednjom verzijom podatka, svesno prihvatajući kratkotrajna zastoja u radu.

Naredni primer ilustruje nivo izolacije **READ COMMITTED** sa parametrom **RECORD_VERSION** koji omogućava sesiji da vidi najnovije potvrđene promene drugih korisnika unutar iste transakcije.

Izvorni kod: Read Committed RECORD_VERSION

```
13 /*sesija A*/
14 SET TRANSACTION ISOLATION LEVEL READ COMMITTED RECORD_VERSION;
15 SELECT E.EMP_NO, E.FIRST_NAME, D.DEPARTMENT, E.SALARY
16 FROM EMPLOYEE E
17 JOIN DEPARTMENT D ON E.DEPT_NO = D.DEPT_NO
18 WHERE E.SALARY > 80000
19 ORDER BY E.SALARY DESC;
20
21 /*sesija B*/
22 UPDATE EMPLOYEE SET SALARY = 7580000 WHERE EMP_NO = 118;
23 COMMIT;
24 /*ponovo izvršiti SELECT iz sesije A*/
```

Za razliku od Snapshot izolacije, READ COMMITTED RECORD_VERSION omogućava sesiji A da u realnom vremenu detektuje sve potvrđene izmene nastale u bazi. Kada sesija B izvrši UPDATE i potvrdi transakciju, ponovljeni upit u sesiji A će odmah prikazati novu vrednost plate, jer ovaj režim rada uvek čita poslednju "komitovanu" verziju zapisa. Ovakav pristup je idealan za

operativno praćenje podataka na replici gde je neophodno imati uvid u najsvežije stanje bez potrebe za ponovnim pokretanjem transakcije.

Sa druge strane, ako bi istu ovu transakciju pokrenuli, ali sa parametrom NO RECORD_VERSION, sistem bi se ponašao restriktivnije u situacijama kada dođe do istovremenog pristupa istom zapisu.

Izvorni kod: Read Committed NO RECORD_VERSION

```
/*sesija A*/
SET TRANSACTION ISOLATION LEVEL READ COMMITTED NO RECORD_VERSION;
SELECT E.EMP_NO, E.FIRST_NAME, D.DEPARTMENT, E.SALARY
FROM EMPLOYEE E
JOIN DEPARTMENT D ON E.DEPT_NO = D.DEPT_NO
WHERE E.SALARY > 80000
ORDER BY E.SALARY DESC;

/*sesija B*/
UPDATE EMPLOYEE SET SALARY = 7580000 WHERE EMP_NO = 118;
COMMIT;
/*ponovo izvršiti SELECT iz sesije A*/
```

U ovom režimu, ukoliko sesija B pokuša da izmeni podatak koji sesija A upravo čita (ili obrnuto), Firebird neće dozvoliti sesiji A da pristupi starijoj verziji zapisa dok se transakcija u sesiji B ne završi. Umesto da odmah dobije rezultat, sesija A će biti "blokirana" i čekaće da sesija B izvrši *commit* ili *rollback*. Ovaj parametar se koristi kada je neophodno osigurati da se čitaju isključivo finalne i stabilne verzije podataka bez korišćenja privremenih verzija zapisa, čime se postiže stroža kontrola nad konkurencijom pristupa, ali po cenu potencijalnog čekanja na oslobađanje resursa.

4.3. Snapshot Table Stability

Snapshot Table Stability (poznat i kao *Consistency*) predstavlja najstroži i najekskluzivniji režim izolacije u Firebird-u, koji je funkcionalni ekvivalent Serializable nivou iz SQL standarda. Kada transakcija zahteva ovaj nivo, ona praktično traži ekskluzivni monopol nad tabelama kojima pristupa. Ovaj režim sprečava bilo koju drugu transakciju da vrši izmene u tabelama koje je Snapshot Table Stability transakcija "rezervisala" za svoj rad.

Iako pruža najviši stepen sigurnosti, ovaj nivo se u modernom višekorisničkom razvoju koristi ekstremno retko i uz veliki oprez. Njegova primena može dovesti do potpune paralize sistema jer blokira rad svih ostalih korisnika nad tim tabelama. Obično se čuva za masovne administrativne operacije, migracije podataka ili kompleksne proračune gde je bilo kakva paralelna aktivnost apsolutno nedopustiva.

Izvorni kod: Snapshot Table Stability

```
/*sesija A*/
SET TRANSACTION ISOLATION LEVEL SNAPSHOT TABLE STABILITY READ
WRITE NO WAIT;

```

```
SELECT COUNT(*) AS BROJ_ZAPOSLENIH FROM EMPLOYEE;
SELECT COUNT(*) AS BROJ_ODJELJENJA FROM DEPARTMENT;

/*sesija B*/
UPDATE EMPLOYEE SET SALARY = 7580000 WHERE EMP_NO = 118;
/*operacija se blokira*/
```

4.4. Read Committed Read Consistency

Jedan od najvećih izazova u starijim verzijama Firebirda bio je problem "izgubljenih ažuriranja" (update conflicts) u Read Committed modu. Firebird 4.0 je uveo revolucionarni režim pod nazivom READ CONSISTENCY.

U ovom režimu, svaki pojedinačni SQL operacija (na primer, jedan UPDATE koji menja hiljade redova istovremeno) dobija svoj sopstveni, privremeni mini-snapshot. To osigurava da je svaka instrukcija konzistentna sama sa sobom tokom celog svog trajanja. Ukoliko tokom izvršavanja tog upita neka druga transakcija potvrdi izmene na istim podacima, Firebird neće prekinuti rad greškom "*deadlock*" ili "*update conflict*". Umesto toga, on će inteligentno i automatski "ponoviti" deo upita kako bi osigurao da su sve promene ispravno primenjene na najnovije, upravo potvrđeno stanje baze. Ovo predstavlja ogroman iskorak za stabilnost aplikacija jer baza preuzima na sebe kompleksan posao sinhronizacije koji su ranije morali ručno da rešavaju programeri kroz komplikovane mehanizme ponavljanja transakcija.

.

Izvorni kod: Read Committed RECORD_VERSION

```
/* sesija A - Read Consistency mod */
SET TRANSACTION ISOLATION LEVEL READ COMMITTED READ CONSISTENCY;

UPDATE EMPLOYEE
    SET SALARY = SALARY * 1.10
    WHERE DEPT_NO = '621';
/* Još se ne commituje */

/* sesija B - menja jedan red iz odeljenja */
UPDATE EMPLOYEE
    SET SALARY = 99999
    WHERE EMP_NO = 28;
COMMIT;

/* sesija A */
COMMIT;

SELECT EMP_NO, SALARY
FROM EMPLOYEE
WHERE DEPT_NO = '621'
ORDER BY EMP_NO;
```

Rezultat: sesija A uspešno završava UPDATE bez greške "update conflict", iako je sesija B u međuvremenu menjala isti red. Firebird automatski ponavlja dio upita i primjenjuje povećanje od 10% na najnoviju potvrđenu vrednost, umesto da prijavi grešku i primorava programera da ponovi čitavu transakciju.

5. Mehanizmi zaključavanja (locking) i rešavanje konflikata

Iako arhitektura Firebirda omogućava čitaocima da nikada ne blokiraju pisce, situacija se drastično menja u momentu kada dve ili više transakcija pokušaju da istovremeno izvrše izmenu nad istim logičkim redom podataka. U takvim situacijama, MGA mehanizam verzija više nije dovoljan da osigura integritet, te na scenu stupaju mehanizmi zaključavanja. Ovi mehanizmi garantuju da u svakom trenutku samo jedna transakcija može imati status "vlasnika" izmene nad određenim resursom čime se sprečava gubitak podataka i narušavanje poslovne logike.

5.1. Pesimističko zaključavanje

U specifičnim poslovnim procesima, programer često ne želi da čeka trenutak fizičkog upisa (UPDATE) da bi saznao da li je određeni podatak slobodan za manipulaciju. Postoje situacije gde je neophodno "rezervisati" red odmah pri samom čitanju, pre nego što korisnik uopšte započne unos novih vrednosti. Za tu svrhu Firebird nudi klauzulu **SELECT ... WITH LOCK.**

Ovaj pristup se naziva **pesimističko zaključavanje** jer se zasniva na pretpostavci da će verovatno doći do konflikta, pa se resurs preventivno zauzima tj. rezerviše. Čim se izvrši ovakav upit, Firebird postavlja ekskluzivni katanac na pročitane redove. Svaka druga transakcija koja pokuša da izvrši **UPDATE** ili **DELETE** nad tim redovima biće trenutno blokirana ili odbijena (zavisno od parametara čekanja) sve dok se prva transakcija ne završi putem *commit* ili *rollback* naredbe.

Na primer, u sistemima za hotelske rezervacije ili prodaju avio-karata, ključno je da u momentu kada korisnik odabere slobodno mesto, niko drugi ne može da mu ga "otme" dok on popunjava svoje lične podatke na ekranu. Upotrebom **SELECT WITH LOCK**, aplikacija rezerviše to sedište u bazi istog trenutka kada ga prikaže korisniku, čime se izbegava situacija da dva putnika plate istu kartu. Naredni primer ilustruje primenu **pesimističkog zaključavanja** pomoću klauzule WITH LOCK

Izvorni kod: Pesimističko zaključavanje

```
SELECT EMP_NO, FIRST_NAME, LAST_NAME, SALARY
FROM EMPLOYEE
WHERE EMP_NO = 2
WITH LOCK;

-- pokrenuti sesiju B
UPDATE EMPLOYEE
SET SALARY = 120000
WHERE EMP_NO = 2;
```

5.2. Optimistički pristup

Standardni i podrazumevani način rada Firebirda je **optimistička kontrola**. Ovde baza dozvoljava korisniku da slobodno čita podatke bez ikakvog zaključavanja, pod pretpostavkom da je verovatnoća konflikta mala. Međutim, prava provera se vrši u "zadnji čas" tj. u momentu kada korisnik pokuša da izvrši **UPDATE**.

Ukoliko je u intervalu između čitanja i pokušaja upisa neka druga transakcija već izmenila i potvrdila (*commit*) izmenu nad tim redom, Firebird će prvoj transakciji izbaciti grešku poznatu kao ***Update Conflict***. Programer u ovom slučaju mora predvideti obradu izuzetaka u kodu — najčešće tako što će ponovo učitati sveže podatke, prikazati ih korisniku uz informaciju o tuđoj izmeni, ili pokušati automatsko spajanje promena.

Izvorni kod: Update With Conflict

```
76 /* sesija A */
77 SET TRANSACTION ISOLATION LEVEL SNAPSHOT;
78 UPDATE EMPLOYEE SET PHONE_EXT = '111' WHERE EMP_NO = 4;
79 /* BEZ COMMIT-A */
80
81 /* sesija B u drugom prozoru*/
82 SET TRANSACTION ISOLATION LEVEL SNAPSHOT;
83 UPDATE EMPLOYEE SET PHONE_EXT = '222' WHERE EMP_NO = 4;
84 -- Transakcija ce da ceka (WAIT)
```

5.3. Arhitektura i uloga Lock Manager-a

Za preciznu koordinaciju svih katanaca, provera i statusa je zadužen **Lock Manager**. To je mehanizam koji vodi rigoroznu evidenciju o svim aktivnim zaključavanjima na nivou celog servera. Njegova implementacija zavisi od arhitekture servera npr. kod *SuperServer* modela, on je integrisan unutar samog procesa, dok kod *Classic* ili *SuperClassic* modela funkcioniše kao poseban entitet ili kroz deljenu memoriju.

Lock Manager koristi deljenu memoriju (engl. *shared memory*) kako bi u realnom vremenu pratio ko drži koji katanac i koji procesi čekaju na red. On ne upravlja samo zaključavanjem redova, već i:

- **Zaključavanjem tabela** – kada se vrše operacije nad strukturom
- **Zaključavanjem stranica baze** – prilikom fizičkih operacija upisa na disk
- **Logičkim objektima** – poput generatora (sekvenci) i eksternih datoteka

Njegova uloga je uloga "saobraćajca" koji održava red u višekorisničkom okruženju. Efikasnost Lock Manager-a je presudna za skalabilnost – ako je loša konfiguracija parametara zaključavanja onda će se on pretvoriti u „usko grlo” pri velikom broju konkurentnih sesija.

5.4. Deadlock u Firebirdu: Algoritmi detekcije i strategije izbegavanja

Deadlock, iliti mrtva petlja, je klasičan problem paralelnog programiranja koji se javlja kada dve ili više transakcija uđu u začarani krug čekanja. Na primer, *transakcija A* zaključa *red 1* i čeka da se oslobodi *red 2*, dok istovremeno *transakcija B* drži *red 2* i čeka na *red 1*. U tom trenutku, oba procesa ostaju trajno zamrznuta jer niko ne može da nastavi rad.

Firebird poseduje sofisticiran ugrađen algoritam za detekciju ovakvih situacija. Lock Manager periodično analizira **graf čekanja (engl. *wait-for graph*)**. Čim detektuje postojanje zatvorene petlje, on interveniše tako što nasilno prekida jednu od transakcija u krugu i šalje joj grešku *„deadlock“*. Time se oslobađaju resursi i omogućava drugoj transakciji da završi svoj posao.

5.5. Parametri čekanja: Wait, No Wait i uticaj Lock Timeout parametra

Kada jedna transakcija naiđe na red koji je već zaključan od strane druge transakcije, njeno dalje ponašanje direktno zavisi od parametara definisanih unutar **TPB (Transaction Parameter Block)** bloka prilikom njenog startovanja:

- **WAIT** – transakcija ulazi u stanje mirovanja i čekaće onoliko dugo koliko je potrebno da se zaključani red oslobodi. Iako ovo deluje logično, u praksi može dovesti do "zamrzavanja" korisničkog interfejsa ako druga transakcija ostane otvorena predugo (npr. korisnik je otišao na pauzu dok mu je otvoren prozor za izmenu).
- **NO WAIT** – transakcija se momentalno odbija uz grešku ukoliko naiđe na bilo kakav katanac. Ovo je idealno za interaktivne aplikacije (GUI) jer korisnik trenutno dobija povratnu informaciju da je podatak zauzet i može nastaviti sa drugim poslom.
- **LOCK TIMEOUT** – predstavlja moderan kompromis uveden u novijim verzijama Firebirda. Transakcija će čekati definisani broj sekundi (npr. 5 ili 10). Ukoliko se u tom roku red ne oslobodi, transakcija prekida čekanje i podiže izuzetak. Ovo sprečava beskonačno čekanje i omogućava sistemu da ostane operativan čak i u ekstremnim situacijama kada konekcije "pucaju" dok drže aktivne lock-ove.

U nastavku je dat primer jednog deadlock-a:

Izvorni kod: Primer deadlock-a

```
/* sesija A - zaključava odjeljenje */
SET TRANSACTION READ WRITE NO WAIT;
UPDATE DEPARTMENT
SET BUDGET = BUDGET * 1.10
WHERE DEPT_NO = '621';

/* sesija B - zaključa zaposlenog, pa pokuša odeljenje*/
SET TRANSACTION READ WRITE NO WAIT;

UPDATE EMPLOYEE
SET JOB_GRADE = 3
WHERE EMP_NO = 36;

```

```
/* sesija A – pokušava da izmeni zaposlenog */
UPDATE EMPLOYEE
SET JOB_GRADE = 4
WHERE EMP_NO = 36;
/* sesija A čeka na sesiju B... */

/* sesija B - pokušava da izmeni odeljenje */
UPDATE DEPARTMENT
SET BUDGET = BUDGET * 0.90
WHERE DEPT_NO = '621';
```

Ovaj primer prikazuje klasičan “deadlock” jer dve sesije blokiraju jedna drugu do potpunog zastoja. Sesija A je rezervisala tabelu odeljenja (**DEPARTMENT**), dok je sesija B istovremeno rezervisala tabelu zaposlenih (**EMPLOYEE**). Problem nastaje kada sesija A pokuša da pristupi podacima koje drži sesija B, a sesija B pokuša da dohvati ono što drži sesija A. Pošto nijedna sesija ne želi da pusti svoj "plen" dok ne dobije onaj drugi, sistem ulazi u neršivu petlju čekanja.

Pored WAIT-a i NO WAIT-a, ima i parametar LOCK TIMEOUT koji predstavlja kompromis između NO WAIT i WAIT pristupa. Transakcija neće odmah prijaviti grešku kao NO WAIT, ali neće ni čekati beskonačno kao WAIT.

Naredni primer prikazuje kako sesija A zaključava red u tabeli PROJECT, dok Sesija B pokušava da ažurira povezane projekte u tabeli EMPLOYEE. Sesija B čeka tačno 5 sekundi, a ako Sesija A ne oslobodi resurs u tom roku, Sesija B prekida izvršavanje uz grešku deadlock.

Izvorni kod: Primer deadlock-a

```
/* sesija A - zaključaj projekat */
SET TRANSACTION READ WRITE WAIT;

UPDATE PROJECT
    SET PROJ_DESC = 'Azurirani opis projekta'
    WHERE PROJ_ID = 'GUIDE';
/* ne commitujemo jer simuliramo da je korisnik otisao na pauzu */

/* sesija B – pokušava izmenu zaposlenog na istom projektu */
SET TRANSACTION READ WRITE WAIT LOCK TIMEOUT 5;

UPDATE EMPLOYEE
    SET JOB_CODE = 'Mngr'
    WHERE EMP_NO = 24;

UPDATE PROJ_DEPT_BUDGET
    SET PROJECTED_BUDGET = PROJECTED_BUDGET * 1.05
    WHERE PROJ_ID = 'GUIDE'
    AND YEAR_BUDGET = 1994;
/* Sesija B čeka 5 sekundi... */
```

```
/* Nakon 5 sekundi dolazi do greške jer je timeout istekao */

/* sesija A - korisnik se vratio, tek sada commituje, ali je kasno
   jer je sesija B već prekinuta*/
COMMIT;
```

6. Održavanje baze i garbage collection

Specifičnost Firebird arhitekture, zasnovane na MGA modelu, ogleda se u tome što sistem nikada fizički ne briše podatke u trenutku izvršenja komande. Svaka UPDATE ili DELETE operacija za sobom ostavlja staru verziju zapisa. Ukoliko se ovi "ostaci" ne uklanjaju sistematski, baza podataka se suočava sa degradacijom performansi. Proces identifikacije i trajnog uklanjanja ovih nepotrebnih generacija podataka naziva se **Garbage Collection (GC)**.

6.1. Problem rasta baze podataka (Database Bloat) usled akumulacije verzija zapisa

U transakcionim sistemima sa visokim intenzitetom izmena, Firebird može generisati hiljade novih verzija zapisa u veoma kratkom vremenskom intervalu. Ako transakcije traju predugo, te verzije moraju ostati fizički prisutne na stranicama podataka jer postoji teoretska šansa da su nekoj od aktivnih transakcija one još uvek potrebne za održavanje konzistentnosti pogleda.

Rezultat ovog procesa je pojava poznata kao **Database Bloat**. Ova pojava znači da fizička veličina fajla baze podataka raste, dok performanse eksponencijalno opadaju. *Engine* je primoran da prilikom svakog upita pretražuje i preskače čitave nizove nevalidnih verzija kako bi rekonstruisao aktuelno stanje što troši dragocene procesorske resurse i povećava broj I/O operacija. Razumevanje mehanizama čišćenja je stoga imperativ za svakog administratora ili programera koji radi sa Firebirdom.

6.2. Analiza transakcionih markera

Rad *Garbage Collector* mehanizma i opšte performanse Firebird baze podataka direktno zavise od četiri ključna transakciona markera. Ovi parametri se čuvaju u zaglavlju baze podataka (*Database Header*) i predstavljaju kontrolne tačke koje *engine* koristi za navigaciju kroz verzije zapisa. Razumevanje ovih vrednosti je od presudnog značaja za dijagnostiku „zagušenja” baze i prekomernog rasta fizičkog fajla.

6.2.1. Definicija markera i njihova uloga

Tipovi markera su sledeći:

- **Next Transaction ID** – predstavlja redni broj (identifikator) koji će biti dodeljen prvoj sledećoj transakciji koja se pokrene. Ovaj broj se konstantno uvećava i služi kao gornja granica hronologije rada baze.
- **Oldest Interesting Transaction (OIT)** – označava najstariju transakciju koja sistemu još uvek predstavlja „interes”. To je najstariji ID transakcije koja je ili još uvek aktivna, ili je

poništena (*rolled back*), ali njene verzije zapisa još uvek nisu fizički očišćene sa stranica baze podataka. OIT je donji prag ispod kojeg čišćenje više ne može da ide.

- **Oldest Active Transaction (OAT)** – predstavlja ID najstarije transakcije koja se trenutno nalazi u statusu *Active*. Ovo je najkritičniji marker jer direktno blokira proces sakupljanja otpada. *Garbage Collector* ne sme da ukloni nijednu verziju zapisa koja je nastala nakon OAT-a jer ta transakcija potencijalno može zahtevati pristup tim podacima u skladu sa svojim nivoom izolacije.
- **Oldest Snapshot Transaction (OST)** – ovaj marker identifikuje najstariju transakciju pokrenutu u *Snapshot* nivou izolacije. Specifičnost ovog markera je u tome što on „zamrzava" stanje cele baze za tu transakciju, sprečavajući uklanjanje svih verzija koje su postojale u trenutku njenog starta.

Zdravstveno stanje baze se meri razlikom (*Gap*) između markera. U idealnim uslovima, ovi brojevi su blizu jedan drugom.

- **Primer 1 (Zdrava baza)** – Next: 10500, OAT: 10490, OIT: 10485 Razlika je mala, što znači da se transakcije brzo završavaju i da *Garbage Collector* uspešno prati tempo rada.
- **Primer 2 (Problematična baza - Stuck Transaction)** – Next: 25000, OAT: 5000, OIT: 4950 Ovde postoji ogroman jaz. Transakcija ID 5000 je otvorena pre 20.000 ciklusa i još nije zatvorena. Sve verzije podataka nastale u tom međuvremenu se nagomilavaju jer ih baza ne sme obrisati. Ovo dovodi do drastičnog pada performansi SELECT upita.

6.2.2. SQL monitoring markera u realnom vremenu

Uvođenjem monitoring tabela (od verzije 2.1), Firebird je omogućio administratorima da vrše inspekciju unutrašnjeg stanja baze podataka koristeći standardnu SQL sintaksu. Ovi podaci se ne čitaju sa diska, već se generišu u realnom vremenu iz kontrolnih struktura u memoriji servera..

Primer 1: Upit za analizu transakcionog „jaza"

Ovaj upit je primarni alat za proveru "zdravlja" baze. Njegov cilj je da identifikuje razliku između najnovije transakcije i onih koje još uvek drže resurse.

Izvorni kod: Upit za analizu transakcionog „jaza"

```
CREATE OR ALTER VIEW VW_MONITOR_HEALTH
AS
SELECT
    MON$NEXT_TRANSACTION AS NEXT_ID,
    MON$OLDEST_TRANSACTION AS OIT,
    MON$OLDEST_ACTIVE AS OAT,
    MON$OLDEST_SNAPSHOT AS OST,
    (MON$NEXT_TRANSACTION - MON$OLDEST_ACTIVE) AS TRANSACTION_GAP
FROM MON$DATABASE;
```

Obrada transakcija, planovi izvršavanja transakcija, izolacija i zaključavanje u Firebird bazi podataka

Tumačenje upita:

- **TRANSACTION_GAP** predstavlja broj verzija zapisa koje *Garbage Collector* ne može da obradi. Ako ovaj broj raste (npr. preko 20.000), to je jasan indikator da performanse baze počinju da opadaju jer *engine* pri svakom čitanju mora da procesira sve te "nepouzdane" verzije.
- Ako je **MON$OLDEST_SNAPSHOT (OST)** znatno manji od NEXT_ID, to ukazuje na prisustvo transakcije sa *Snapshot* izolacijom koja "drži" staru sliku baze i sprečava čišćenje čak i ako su ti podaci u međuvremenu promenjeni

Primer 2: Upit za identifikaciju blokirajućih procesa

Kada prethodni upit pokaže veliki jaz, neophodno je identifikovati konkretnu sesiju koja ga uzrokuje. Sledeći upit povezuje transakcije sa sesijama kako bi se dobili podaci o fizičkom korisniku.

Izvorni kod: Upit za identifikaciju blokirajućih procesa

```
CREATE OR ALTER VIEW VW_MONITOR_STUCK_USER
AS
SELECT
    A.MON$USER AS STUCK_USER,
    A.MON$REMOTE_ADDRESS AS IP_ADDRESS,
    A.MON$REMOTE_PROCESS AS APP,
    T.MON$TIMESTAMP AS TIME_START,
    T.MON$ISOLATION_MODE AS ISOLATION_LEVEL
FROM MON$TRANSACTIONS T
JOIN MON$ATTACHMENTS A ON T.MON$ATTACHMENT_ID =
A.MON$ATTACHMENT_ID
WHERE T.MON$TRANSACTION_ID = (SELECT MON$OLDEST_ACTIVE FROM
MON$DATABASE);
```

Tumačenje upita:

- *MON$REMOTE_PROCESS* – prikazuje ime izvršnog fajla (npr. moj_program.exe ili isql.exe). Ovo omogućava administratoru da utvrdi da li je problem u određenom modulu aplikacije koji "zaboravlja" da zatvori transakciju.
- *MON$TIMESTAMP* – omogućava uvid u to koliko dugo je transakcija aktivna. Svaka transakcija koja traje duže od nekoliko minuta u OLTP (*Online Transaction Processing*) sistemima smatra se rizičnom.
- *MON$ISOLATION_MODE* – označava kod nivoa izolacije, a značenje koda možete pronaći u tablici ispod.

Vrednost	Izolacija	Objašnjenje
0	Consistency	Najstroža izolacija – vidi samo stanje baze na početku transakcije
1	Concurrency (Snapshot)	Standardni **snapshot** – najčešće korišćen
2	Read Committed (Record Version)	Čita poslednju commitovanu verziju reda
3	Read Committed (No Record Version)	Čeka ako je red zaključan
4	Read Committed Read Consistency	Noviji snapshot za read committed

Tabela 2: Vrednosti MON$ISOLATION_MODE

Analiza putem MON$ tabela nije samo informativna; ona direktno oslikava (ne)efikasnost Garbage Collector-a. U Firebird arhitekturi, GC konstantno prati vrednost *MON$OLDEST_ACTIVE* (OAT). Sve verzije zapisa čiji je Transaction ID manji od OAT-a, a veći od OIT-a, postaju primarna meta za čišćenje.

Međutim, čim SQL monitoring pokaže da TRANSACTION_GAP raste, to je direktan dokaz da je Garbage Collector "blokiran". On fizički ne može da pristupi stranicama baze i oslobodi prostor jer mu pravila MGA arhitekture nalažu da čuva sve verzije koje bi najstarija aktivna transakcija (identifikovana u upitu VW_MONITOR_STUCK_USER) potencijalno mogla da zatraži. Dakle, SQL monitoring nam omogućava da vidimo "uskim grlima" GC-a pre nego što ona dovedu do kritičnog pada performansi.

6.3. Strategije čišćenja: Cooperative, Background i Sweeping (Backsweep) procesi

Firebird implementira tri različite strategije za neutralisanje nepotrebnih verzija zapisa, prilagođene različitim scenarijima opterećenja:

- **Cooperative Garbage Collection –** ovo je najrasprostranjeniji vid čišćenja koji se odvija "reaktivno". Dešava se tokom redovnog rada kada korisnički SELECT upit naiđe na lanac verzija. Dok *engine* čita podatke, on ujedno vrši i proveru kojom identifikuje verzije koje su starije od najstarije aktivne transakcije i uklanja ih. Na ovaj način, sami korisnici kroz svoje upite doprinose održavanju baze podataka.
- **Background Garbage Collection** – karakterističan za modernije verzije (posebno u *SuperServer* modelu), ovaj mehanizam koristi posebnu programsku nit (thread) koja u pozadini, nezavisno od korisničkih aktivnosti, skenira stranice i vrši čišćenje. Time se primarni korisnički upiti rasterećuju posla oko održavanja lanca verzija.
- **Sweeping (Backsweep)** – ovo je najtemeljniji process i on predstavlja sistematsko skeniranje celokupne baze podataka. *Sweep* identifikuje sve nepotrebne verzije, oslobađa prostor i ažurira status na TIP stranicama. Može se konfigurisati da se pokreće automatski (nakon definisanog broja transakcija, tzv. *Sweep Interval*) ili ručno putem gfix alata. Iako privremeno usporava bazu, neophodan je za "resetovanje" indikatora zdravlja baze.

Da bismo demonstrirali rad Garbage Collector-a, potrebno je prvo generisati akumulirane verzije zapisa. Izvršavanjem pet uzastopnih UPDATE operacija nad istim redom, Firebird kreira pet novih fizičkih verzija tog zapisa na stranicama baze, pri čemu svaka prethodna verzija ostaje netaknuta kao "back version". Nakon svakog COMMIT-a, stara verzija postaje kandidat za čišćenje, ali Garbage Collector možda još nije stigao da je ukloni. Merenjem GC_GAP vrednosti **pre**pokretanja sweep procesa možemo videti koliko verzija zapisa čeka na čišćenje. Što je GC_GAP veći, to znači da Garbage Collector zaostaje za tempom izmena i da se na stranicama baze nakuplja sve više starih verzija koje usporavaju SELECT upite jer ih *engine* mora preskakati pri svakom čitanju.

Izvorni kod: Generisanje verzija i merenje GC GAP-a

```
SET TRANSACTION READ WRITE;
UPDATE EMPLOYEE SET PHONE_EXT = '001' WHERE EMP_NO = 2;
COMMIT;
UPDATE EMPLOYEE SET PHONE_EXT = '002' WHERE EMP_NO = 2;
COMMIT;
UPDATE EMPLOYEE SET PHONE_EXT = '003' WHERE EMP_NO = 2;
COMMIT;
UPDATE EMPLOYEE SET PHONE_EXT = '004' WHERE EMP_NO = 2;
COMMIT;
UPDATE EMPLOYEE SET PHONE_EXT = '005' WHERE EMP_NO = 2;
COMMIT;

/*merenje GAP-a pre sweeping-a*/
SELECT
    MON$NEXT_TRANSACTION    AS NEXT_TID,
    MON$OLDEST_ACTIVE       AS OAT,
    MON$OLDEST_TRANSACTION  AS OIT,
    MON$NEXT_TRANSACTION - MON$OLDEST_ACTIVE AS GC_GAP
FROM MON$DATABASE;
```

Pokretanjem ručnog sweep procesa iz komandne linije:

Izvorni kod: Generisanje verzija i mjerenje GC GAP-a

```
gfix -sweep -user SYSDBA -password masterkey "C:\Program
Files\Firebird\Firebird_5_0\examples\empbuild\employee.fdb"
```

A zatim ponovnim merenjem GC_GAP vrednosti, možemo potvrditi da je Garbage Collector uspešno uklonio nakupljene verzije.

7. Planovi izvršavanja (Execution Plans) i monitoring

U Firebirdu, planovi izvršavanja nisu statični; oni su duboko povezani sa stanjem verzija zapisa i nivoom izolacije u kojem se upit pokreće.

7.1. Uloga TPB (Transaction Parameter Block) u donošenju odluka

Svaka interakcija između klijentske aplikacije (npr. Delphi aplikacije preko FireDAC drajvera) i servera započinje definisanjem **TPB** strukture. Ovo je binarni protokol kojim aplikacija diktira uslove pod kojima će se upiti izvršavati. Optimizer koristi ove parametre kao primarne ulazne podatke za sastavljanje plana:

- **Access Mode (Read Only vs. Read Write)** - ukoliko je transakcija deklarisana kao *Read Only*, Optimizer eliminiše sve rutine koje se tiču zaključavanja i pripreme za potencijalni upis, što značajno rasterećuje CPU.
- **Isolation Level** – je nivo izolacije *Snapshot Table Stability*, Optimizer dobija informaciju da se podaci u tabelama neće menjati dok transakcija traje. To mu omogućava da planira pristupe bez straha od paralelnih izmena, često birajući agresivnije algoritme za spajanje tabela (*Merge Joins*).

7.2. Uticaj nivoa izolacije na izbor putanje

U Firebird arhitekturi, indeksi su takođe podložni MGA mehanizmu. Svaki ključ u indeksu mora biti proveren naspram stanja vidljivosti transakcije. Ovde nastaje korelacija između "prljave" baze i sporih upita:

1. **Problem akumuliranih verzija** – ako tabela ima visok nivo *bloat-a* (zbog niske vrednosti OAT markera), indeksi postaju pretrpani pokazivačima na verzije koje više nisu validne, ali još nisu očišćene.
2. **Snapshot kazna –** u *Snapshot* izolaciji, upit koji koristi indeks nad tabelom sa mnogo verzija može biti sporiji nego što se očekuje. Razlog je taj što motor baze mora za svaki ključ u indeksu da izvrši proveru na **TIP stranici** kako bi utvrdio da li je ta konkretna verzija vidljiva.
3. **Povlačenje u "Natural Scan" -** u ekstremnim slučajevima, kada Optimizer detektuje da je broj nevalidnih verzija u indeksu previsok, on može doneti odluku da potpuno ignoriše indeks i uradi **NATURAL scan** (čitanje cele tabele). Ovo objašnjava situacije gde isti upit radi trenutno na test bazi, dok u produkciji sa mnogo korisnika "zamrzava" aplikaciju.

Ovaj primer analizira kako različiti nivoi izolacije utiču na performanse i način na koji Firebird engine planira izvršavanje istog upita.

Izvorni kod: Uticaj izolacije na execution plan

```
/* nivo izolacije snapshot read only */
SET TIME ON;
SET TRANSACTION ISOLATION LEVEL SNAPSHOT READ ONLY;
SET STATS ON;
SET PLAN ON;
```

```

SELECT E.EMP_NO,
      E.FIRST_NAME,
      E.LAST_NAME,
      D.DEPARTMENT,
      E.SALARY
FROM EMPLOYEE E
JOIN DEPARTMENT D ON E.DEPT_NO = D.DEPT_NO
WHERE E.SALARY > 80000
ORDER BY E.SALARY DESC;

/* nivo izolacije READ COMMITTED RECORD VERSION*/
SET TIME ON;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED RECORD_VERSION;
SET STATS ON;
SET PLAN ON;

SELECT E.EMP_NO,
      E.FIRST_NAME,
      E.LAST_NAME,
        D.DEPARTMENT,
      E.SALARY
FROM EMPLOYEE E
JOIN DEPARTMENT D ON E.DEPT_NO = D.DEPT_NO
WHERE E.SALARY > 80000
ORDER BY E.SALARY DESC;
```

Kao rezultat prethodna dva upita, date su uporedne performanse oba plana:

SNAPSHOT READ ONLY;	READ COMMITTED RECORD_VERSION
Current memory = 3633328 Delta memory = 1920 Max memory = 3712448 Elapsed time = 0.006 sec Buffers = 256 Reads = 0 Writes = 0 Fetches = 140	Current memory = 3786976 Delta memory = 26144 Max memory = 3867360 Elapsed time = 0.010 sec Buffers = 256 Reads = 21 Writes = 0 Fetches = 235

Analiza rezultata: Snapshot je pokazao brže izvršavanje od Read Committed, što je očekivano za malu bazu bez konkurentnih sesija i akumuliranih verzija zapisa. U produkcijskom okruženju sa visokim brojem paralelnih transakcija, Read Committed bi pokazao bolje performanse zbog kraćih lanaca verzija kroz koje Optimizer mora da prolazi.

7.3. Dijagnostika transakcija u realnom vremenu putem sistemskih tabela (MON$TRANSACTIONS)

Za razliku od prethodno opisanih monitoringa markera, ovde nas zanima šta se tačno dešava "ispod haube" dok se upit izvršava. Tabela **MON$STATEMENTS** nam omogućava da vidimo plan koji je Optimizer izabrao za svaki upit koji je trenutno u toku.

Naredni primer predstavlja napredni nivo administracije gde koristimo sistemski monitoring za dijagnostiku performansi u realnom vremenu

Izvorni kod: SQL primer za detekciju sporih upita i njihovih planova

```
CREATE OR ALTER VIEW VW_MONITOR_ACTIVE_PLANS
AS
SELECT
    A.MON$USER AS ACTIVE_USER,
    T.MON$TRANSACTION_ID AS TID,
    T.MON$ISOLATION_MODE AS LEVEL_ISOLATION,
    ST.MON$SQL_TEXT AS SQL_STATEMENT,
    ST.MON$EXPLAINED_PLAN AS EXECUTION_PLAN,
    ST.MON$TIMESTAMP AS START_TIME
FROM MON$STATEMENTS ST
JOIN MON$TRANSACTIONS T ON ST.MON$TRANSACTION_ID =
T.MON$TRANSACTION_ID
JOIN MON$ATTACHMENTS A ON T.MON$ATTACHMENT_ID =
A.MON$ATTACHMENT_ID
WHERE ST.MON$STATE = 1; --upiti koji se trenutno izvršavaju
```

Ključna vrednost ovog pristupa leži u detekciji neefikasnih operacija; pojavljivanje termina NATURAL u planu izvršavanja signalizira da sistem ignoriše indekse i vrši sekvencijalno čitanje cele tabele. To je najčešće posledica nakupljenih starih verzija zapisa koje nivoi izolacije poput SNAPSHOT moraju da uzmu u obzir, čime se još jednom potvrđuje da pravilno upravljanje transakcijama i redovno čišćenje baze direktno diktiraju brzinu odziva celokupnog sistema.

8. Zaključak

Istraživanje sprovedeno u okviru ovog rada potvrdilo je da je Firebird, iako se često svrstava u kategoriju "lakših" relacionih baza podataka, zadržao jedan od najsofisticiranijih i najdoslednijih transakcionih modela u industriji. Specifičan put koji je ovaj sistem izabrao kroz implementaciju **Multi-Generational Architecture (MGA)** predstavlja inženjersko rešenje koje na elegantan način balansira između strogih teoretskih zahteva ACID principa i praktičnih potreba za visokim performansama u višekorisničkom radu.

Ključni zaključak analize unutrašnje strukture Firebirda je da njegova snaga leži u eliminaciji konflikata između čitača i pisača. U svetu gde se zahtevi za generisanjem izveštaja u realnom vremenu često preklapaju sa intenzivnim unosom podataka, Firebird-ov MVCC (Multi-Version Concurrency Control) model nudi prednost koju je teško nadmašiti tradicionalnim mehanizmima zaključavanja. Činjenica da dugačke `Snapshot` transakcije mogu da čitaju konzistentno stanje baze bez postavljanja ijednog katanca na operativne tabele, omogućava razvoj sistema koji su istovremeno robusni i responsivni.

Međutim, praktični testovi sprovedeni u radu jasno su ukazali na to da ova arhitektonska sloboda nosi sa sobom i specifične inženjerske odgovornosti. **Garbage Collection** mehanizam, iako automatizovan, nije svemoćan. On direktno zavisi od discipline klijentskih aplikacija i ispravnog upravljanja životnim ciklusom transakcija. Nagomilavanje starih verzija zapisa, uzrokovano "zaboravljenim" ili predugačkim transakcijama, identifikovano je kao najveći rizik po performanse sistema. Iz toga proističe kritičan zaključak: uspeh implementacije Firebird baze podataka ne zavisi samo od dizajna šeme i kvaliteta upita, već primarno od kontrole transakcionih markera poput `OAT` i `OIT`.

Takođe, evolucija sistema kroz verziju 4.0 i uvođenje **Read Committed Read Consistency** nivoa izolacije pokazuje da zajednica programera aktivno radi na rešavanju najveće mane optimističkog modela – učestalih konflikata pri upisu. Ovim dodatkom, Firebird se približio modernim „cloud-scale” zahtevima, gde se očekuje da baza sama rešava sitne sukobe na nivou instrukcije, smanjujući potrebu za kompleksnom logikom ponavljanja transakcija unutar samog aplikativnog koda.

Na kraju, može se konstatovati da Firebird transakcioni model zahteva "promenu paradigme" kod inženjera koji dolaze iz svetova gde dominira strogo zaključavanje. On zahteva proaktivno praćenje zdravlja baze kroz sistemske monitoring tabele i pažljivo biranje parametara unutar **Transaction Parameter Block-a**. Na kraju, možemo reći da Firebird predstavlja izuzetan primer kako se sofisticirana arhitektura može iskoristiti za postizanje maksimalne konkurentnosti, pod uslovom da se razumeju i poštuju pravila po kojima funkcioniše njegov "mehanizam verzija". Budućnost ovog sistema, sudeći po trendovima u novijim verzijama, ide ka još većoj automatizaciji održavanja, čime će se dodatno smanjiti jaz između njegove unutrašnje kompleksnosti i lakoće korišćenja u produkcionim uslovima.

9. Literatura

[1] **ANSI/ISO/IEC 9075-2:2016**. Information technology – Database languages – SQL – Part 2: Foundation (SQL/Foundation). (Međunarodni standard za SQL nivoe izolacije).

[2] **Borrie, H. (2004).** The Firebird Book: A Reference for Database Developers. Apress. (Osnovna literatura za unutrašnju arhitekturu i MGA mehanizam).

[3] **Firebird Project.** Firebird 4.0 Language Reference. Zvanična dokumentacija dostupna na: firebirdsql.org.

[4] **Firebird Project (2021).** Firebird 4.0 Release Notes: Monitoring Enhancements and Read Consistency. (Tehnička dokumentacija za verziju 4.0+).

[5] **Firebird SQL.** Transaction Inventory Page (TIP) internals. Tehnički članci o unutrašnjoj strukturi stranica inventara transakcija.

[6] **Kuznetsov, D.** Understanding Firebird Garbage Collection. Tehnička dokumentacija o mehanizmima čišćenja i održavanja baze podataka.

[7] **Sistemske tabele (Monitoring RDB$ i MON$)** – Interna dokumentacija metapodataka Firebird engine-a.